

BREAKOUT MASTER CHALLENGE

Name of author

CSE SOCIETY,DHANBAD

ABSTRACT

This project attempts to utilize deep reinforcement learning (RL) Models to play the Atari game Breakout [1], using it as a benchmark for evaluating decision-making algorithms for various models in a high-dimensional visual environment. we use Arcade learning environment(ALE)[2] via the gymnasium interface to simulate the breakout environment for our agent.using algorithms implementations like Deep Q-Learning(DQN)[3] comparing it in many metrics like average score ,increase in score w.r.t episodes,average error ,gameplay strategies,etc.

1. INTRODUCTION

Reinforcement learning (RL) has become a strong paradigm for training agents to make sequential decisions in complex environments. Within the collection of benchmarks for testing RL algorithms, Atari games have enjoyed a pivot-al role because they provide rich, high-dimensional visual inputs and well-specified reward structures. Breakout, specifically, is a challenging environment where agents must learn good strategies for maximizing long-term rewards based on pixel data alone. Within this paper, we aim to rigorously assess and compare the efficiency of these deep RL algorithms in the context of Atari game Breakout. Using the *Arcade Learning Environment (ALE)* via the gymnasium in- terface, we train and deploy agents with DQN using different techniques like *FRAME STACKING* and *PRIORITIZED EXPERIENCE RE-PLAY(PER)*, measuring their performance on a number of important metrics such as average score, learning progress, error rates, and game playing strategies.

2. BACKGROUND

Atari game breakout gives pixel data as output and accepts 4 discrete actions that are moveLeft,moveRight ,doNothing and fire(for breakout fire = doNothing). an **observation** is whatever the agent can see i.e whatever the inputs of our model are .

A **state** represents the complete set of variables upon which the next action chosen by the model depends. Formally, if $a^* = f(S)$ and there exists a variable $x \notin S$ such that $a^* = f(S, x)$, then S is incomplete. Thus,

S is complete when:

$$\nexists x \notin S \text{ such that } a^* = f(S, x).$$

Breakout is an sparse reward environment that means most of the time reward an agent gets is zero there are many papers that solves this problem[4][5][6] one of the techniqe to solve this is using **DQN** we compute a **value function** that estimates the cumulative future rewards. This helps guide the agent toward states that are expected to yield higher rewards in the future.

$$R = r_1 + r_2 + r_3 + \dots + r_n = \sum_{i=1}^n r_i$$

$$R_t = r_t + r_{t+1} + \dots + r_n = \sum_{k=t}^n r_k$$

It is preferable for the agent to receive rewards *as soon as possible*. so we use **discounted-value function** for this, defined as:

$$R = r_1 + \lambda r_2 + \lambda^2 r_3 + \dots + \lambda^{n-1} r_n = \sum_{i=1}^n \lambda^{i-1} r_i$$

$$R_t = r_t + \lambda r_{t+1} + \lambda^2 r_{t+2} + \dots + \lambda^{n-t} r_n = \sum_{k=t}^n \lambda^{k-t} r_k$$

$$\Rightarrow R_t = r_t + \lambda R_{t+1}$$

where λ is **discount factor** $\lambda \in [0, 1]$.

We define the **action-value function** (also called the **Q-function**) as $Q(S, a)$, which represents the maximum expected discounted future reward achievable when the agent performs action a in state S , and thereafter follows the optimal policy.

$$Q(S, a) = \max R_{t+1}$$

we define **policy** π defines the mapping from states to actions. Genrally –

$$\pi(S) = \arg \max_a Q(S, a)$$

We can compute the **Q-function** using the **Bellman equation**. [7]

$$Q(S, a) = r + \lambda \max_{a'} Q(S', a')$$

Algorithm 1 Q-Learning Algorithm

- 1: initialize $Q[num_states, num_actions]$ arbitrarily
 - 2: observe initial state s
 - 3: **repeat**
 - 4: select and carry out an action a
 - 5: observe reward r and new state s'
 - 6: $Q[s, a] = Q[s, a] + \alpha(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$
 - 7: $s = s'$
 - 8: **until** terminated
-

In practice, Value Iteration is impractical

- Very limited states/actions
- Cannot generalize to unobserved states

so we use a **function approximator** $Q^*(S, a; w)$ to estimate the optimal Q-function.

The **loss** function is defined as the expected squared difference between the target value and the predicted Q-value:

$$L = \mathbb{E} \left[(r + \lambda \pi^*(S') - Q^*(S, a; w))^2 \right]$$

where $\pi^*(S') = \arg \max_{a'} Q^*(S', a'; w)$ is the policy induced by the current Q-function,

we introduce the **epsilon-greedy policy** (ϵ -greedy policy) to stop agent from becoming too greedy i.e. choose actions that it thinks is best always.

$$\pi_\epsilon(S) = \begin{cases} \text{a random action } a \in \mathcal{A}, & \text{with probability } \epsilon, \\ \arg \max_a Q(S, a), & \text{with probability } 1 - \epsilon. \end{cases}$$

ϵ is gradually decreased — encouraging the agent to exploit what it has learned while still occasionally exploring.

This process allows the agent to achieve a balance between **Exploration**(random guessing) and **Exploitation**(choose the best one according to the Q-value)

we use **Experience replay** To remove correlations, in which we build data-set from agent's own experience.

1. Take an action a_t according to the ϵ -greedy policy.
2. During gameplay, store each transition $\langle S_t, a_t, r_{t+1}, S_{t+1} \rangle$ in a replay memory D .
3. Sample a random mini-batch of transitions $\langle S, a, r, S' \rangle$ from D .
4. Optimize the Mean Squared Error (MSE) between the predicted Q-values from the Q-network and the Q-learning target values.

$$L = \mathbb{E}_{(S, a, r, S') \sim D} \left[(y - Q(S, a; w))^2 \right]$$

where y is the max discounted reward agent is getting using its current policy.

A **prioritized Experience Replay(PER)** [8] uses TD error as the weights to sample the transitions from D .

$TD_{error} = abs(y - Q(S, a; w))$, the more TD_{error} a transition have the more the agent can learn from it moving forward.

Algorithm 2 Deep Q-learning with Experience Replay

- 1: Initialize replay memory D to capacity N
- 2: Initialize action-value function Q with random weights θ
- 3: Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
- 4: **for** episode = 1, M **do**
- 5: Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
- 6: **for** $t = 1, T$ **do**
- 7: With probability ϵ select a random action a_t
- 8: otherwise select $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$
- 9: Execute action a_t in emulator and observe reward r_t and image x_{t+1}
- 10: Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
- 11: Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
- 12: Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
- 13: Set target

$$y_j = \begin{cases} r_j & \text{if episode terminates,} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise.} \end{cases}$$

- 14: Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$
 - 15: Every C steps reset $\hat{Q} = Q$
 - 16: **end for**
 - 17: **end for**
-

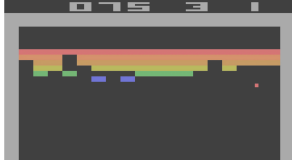
3. METHODOLOGY

we use the "ALE/Breakout-v5" gymnasium environment to train our agent. when doing DQN there is a chance that the agent never converges to any params as the max discounted reward agent is getting using its current policy(y) is dependent upon the agent itself which can make it do back and forth for some time before achieving best weights to escape the loop making the agent inefficient. therefore we consider two models one the **policy network** and other the **target network**, we calculate y using the target network

$$y = r + \lambda \max_a (Q_{target}(S', a))$$

$$L = \mathbb{E}_{(S,a,r,S') \sim D} \left[(y - Q_{policy}(S, a; w))^2 \right]$$

after some regular interval of frames we update $Q_{target} = Q_{policy}$
if you look at the bottom image it is imposible to tell from which angle the ball arrived to this frame ,it's same for the agent also it's impossible for the agent to accurately predict the best action if it's observation is just this frame.



to overcome this issue we use **frame stacking** – a state of an agent is defined not by one frame but a sequence of frames. This allows the agent to get an idea of motion of the ball and to accurately predict the best action for this state.

Table 1: Hyperparameters used for training the DQN agent

Hyperparameter	Description
Target Update	Frames after which Q_{target} is updated(100)
Replay Memory Size	Maximum size of replay buffer $D(100,000)$
Batch Size	Size of batch sampled from D per update(64)
Learning Rate	Step size for optimizer(0.00001)
Epochs	Number of training episodes(35000)
Discount Factor (λ)	Future reward discount factor(0.99)
Epsilon Start	Initial exploration rate(1.0)
Epsilon End	Minimum exploration rate(0.1)
Frame Stack Size	Number of frames stacked per observation(4)

we first use some preprocessing to convert our atari game pixels to grayscale and compress them to 86X86 pixels and stack some last frames one over other in my case 4. then we pass it to a model which uses **CNN** to first convert this raw pixel data to some meaningful data and inputs it to a DQN model which predicts 4 Q-values for the four different actions in atari game.

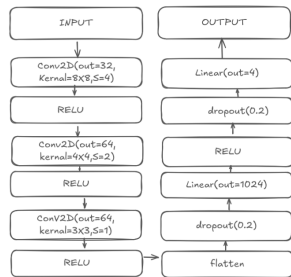


Fig. 1: DQN model

to motivate agent to hit the ball at the starting stages we define a **penalty** of -1 whenever an agent misses a ball.

4. RESULTS AND ANALYSIS

we first used DQN with no frame stacking to check wheather frame stacking has any impact on agent's decision making skills.

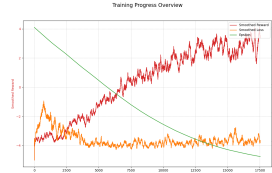


Fig. 2: DQN without Frame Stacking

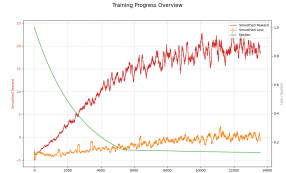


Fig. 3: DQN with Frame Stacking

note that the scale of both the graphs are not same and error is multiplied by 10 for both the graphs. we can clearly see that frame stacking drastically improves the learning of an agent.

after this i found out about the **Dueling Network Architecture**[9],it uses a **state-value function** $V(S)$, which estimates how good it is to be in a given state S .

$$Q_{duel}(S, a) = V(S) + \left(A(S, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(S, a') \right)$$

$$\Rightarrow Q_{duel}(S, a) = Q_{normal}(S, a) - \text{mean}(Q_{normal}(S)) + V(S)$$



Fig. 5: DDQN without PER



Fig. 6: DDQN with PER

Although it doesn't performed well on my particular

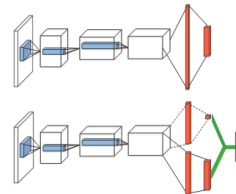


Fig. 4: A popular single stream Q-network (top) and the dueling Q-network (bottom). The dueling network has two streams to separately estimate (scalar) state-value and the advantages for each action; the green output module implements equation (9) to combine them. Both networks output Q-values for each action.

environment while implementing it the papers used a Priority experience replay(PER) for storing the transitions for this. which drastically improved it's performance. that gave me an idea on trying to implement PER in my own DQN model after doing this and training the model for roughly 15 hours. I got the model as

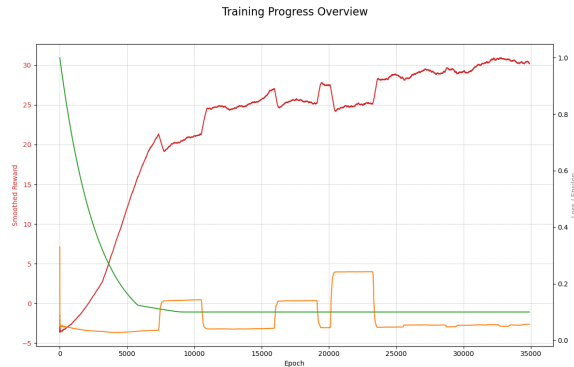


Fig. 7: DQN(using PER) graph (smoothed)

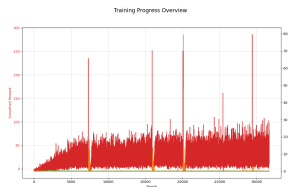


Fig. 8: DQN(using PER) graph (unsmoothed)

a lot of things to unpack here first of all why do the graph has these craters in between well this is because of PER and how the reward function for atari game is made, In the game of atari the bottom two layer of blocks have a reward of 1 while the upper layers have much more reward the agent is converging to a strategy known as **tunneling**.

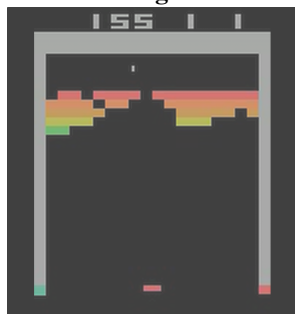


Fig. 9: Agent prioritizing the ball to go one lane where it can tunnel to the back of the game getting much more rewards than usual this strategy is known as **tunneling** whenever the agent performs a successful tunneling action it get so much reward so quickly than what the agent has learned to predict so the agent's TD error increases drastically making the agent learn from these moves a lot compared to others. this is also the

reason why the error is so much during this stage the agent is adjusting it's weights to adapt to this high rewards. this is why we only saw the dip in reward per epochs at the start where the agent is exploring and as the agent adapts to predict this high level of rewards it no longer dips when achieving high rewards which can be seen in later epochs.

Table 2: Performance comparison of different DQN variants on Breakout

Model	Average Reward / Episode	Average Steps / Episode	Observed Behavior
DQN	11.3	165.9	Exhibits highly spontaneous motion with frequent and abrupt changes in direction.
DQN + Frame Stacking	52.8	1166.5	Shows smoother and more continuous motion; tends to clear one row at a time until the third row, then begins tunneling.
DQN + Frame Stacking + PER	113.8	1419.5	Displays early tunneling behavior and continuous motion; demonstrates improved stability and strategic play.

5. CONCLUSION

In this paper, we explored the application of deep reinforcement learning algorithms to the Atari game *Breakout*, using DQN as the primary framework. Our

experiments demonstrated the critical impact of preprocessing techniques such as frame stacking and prioritized action replay(PER) which significantly improved the agent's ability to infer motion and make better decisions.

Through extensive training and evaluation, we observed that combining these techniques enables the agent to achieve higher scores, exhibit advanced gameplay strategies, and reduce learning instability. The results highlight the importance of algorithmic design choices for achieving optimal performance in high-dimensional, discrete-action environments. there are more algorithms like PPO[10] and Rainbow which could prove to be better and I will try to implement them later.

6. REFERENCES

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan

- Wierstra, and Martin Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [2] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling, “The arcade learning environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.
- [3] Christopher J. C. H. Watkins and Peter Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992, Technical Note.
- [4] Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z. Leibo, David Silver, and Koray Kavukcuoglu, “Reinforcement learning with unsupervised auxiliary tasks,” *arXiv preprint arXiv:1611.05397*, 2017.
- [5] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell, “Curiosity-driven exploration by self-supervised prediction,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, pp. 16–17.
- [6] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems (NeurIPS) 30*, 2017, pp. 5040–5050.
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [8] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver, “Prioritized experience replay,” in *International Conference on Learning Representations (ICLR) Workshop Track*, 2015.
- [9] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas, “Dueling network architectures for deep reinforcement learning,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, New York, NY, USA, 2016, pp. 1995–2003, PMLR.
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.